Experiment No.	7
Experiment Title	Dimensionality Reduction and Model Evaluation With and Without PCA
GitHub Repository	https://github.com/sangee-3010/Machine_Learning
Student Name	Sangeetha S K
Register Number	3122247001054
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	30/08/2026

1 Objective

To study the effect of dimensionality reduction using Principal Component Analysis (PCA) on the performance of various machine learning classifiers, by:

a) Training and validating models without PCA (original 30-feature space).

b) Training and validating models with PCA (reduced feature space, 95% variance retained).

For both cases, hyperparameters were tuned, 5-fold cross-validation was applied, and performance was recorded and compared.

2 Dataset

- **Source:** UCI Machine Learning Repository – Breast Cancer Wisconsin (Diagnostic) Dataset (WDBC)
- **Samples:** 569
- **Features:** 30 numerical attributes (`ID` and `Diagnosis` dropped from the feature matrix X)
- **Target classes:** 0 = Benign (357 samples, 62.74%), 1 = Malignant (212 samples, 37.26%)
- **Missing values:** 0
- **Train-test split:** 80/20, stratified – 455 training samples (285 Benign / 170 Malignant), 114 test samples (72 Benign / 42 Malignant)
- **Preprocessing:** `Diagnosis` mapped `{B: 0, M: 1}`; no categorical feature encoding needed (all 30 features already numeric); all features standardized with `StandardScaler` (fit on training data only, applied to both train and test) before either the No-PCA or With-PCA pipelines.

3 Models Evaluated

1. Support Vector Machine (SVM)
2. Naïve Bayes

3. k-Nearest Neighbors (KNN)
4. Logistic Regression
5. Decision Tree
6. Random Forest
7. AdaBoost
8. Gradient Boosting
9. XGBoost
10. Stacking (base learners: Logistic Regression, SVM, KNN; meta-learner: Logistic Regression)

4 Procedure

1. Preprocess and standardize features (`StandardScaler`, fit on training data only).
2. Apply PCA, retaining the number of components needed for 95% cumulative explained variance.
3. For each model, define a hyperparameter search grid.
4. Train and validate each model under both No-PCA and With-PCA settings.
5. Use 5-fold stratified cross-validation (optimizing F1-score) and record results.
6. Compare and analyze whether PCA improved accuracy, F1-score, or stability.

5 PCA Summary

Table 1: PCA Design Choice

Variance target selected	95%
Original number of features	30
Components required for 95% variance	10
Variance explained by selected components	95.21%

Justification: PCA was used to reduce dimensionality while retaining at least 95% of the information present in the original 30-feature space, giving a 3× reduction (30 → 10 features) with minimal information loss (4.79%).

Table 2: Table 1: PCA Variance Explained (all 30 components)

Component	Individual Variance (%)	Cumulative Variance (%)
1	44.59	44.59
2	18.55	63.14
3	9.58	72.72
4	6.59	79.32
5	5.62	84.94
6	3.99	88.93
7	2.21	91.14
8	1.61	92.76
9	1.28	94.04
10	**1.17**	**95.21 (target reached)**
11	1.01	96.22
12	0.91	97.12
13	0.79	97.91
14	0.48	98.39
15	0.30	98.70
16	0.25	98.95
17	0.20	99.15
18	0.17	99.32
19	0.16	99.48
20	0.11	99.58
21	0.10	99.68
22	0.08	99.76
23	0.08	99.84
24	0.05	99.89
25	0.05	99.94
26	0.03	99.97
27	0.02	99.99
28	0.01	100.00
29	0.00	100.00
30	0.00	100.00

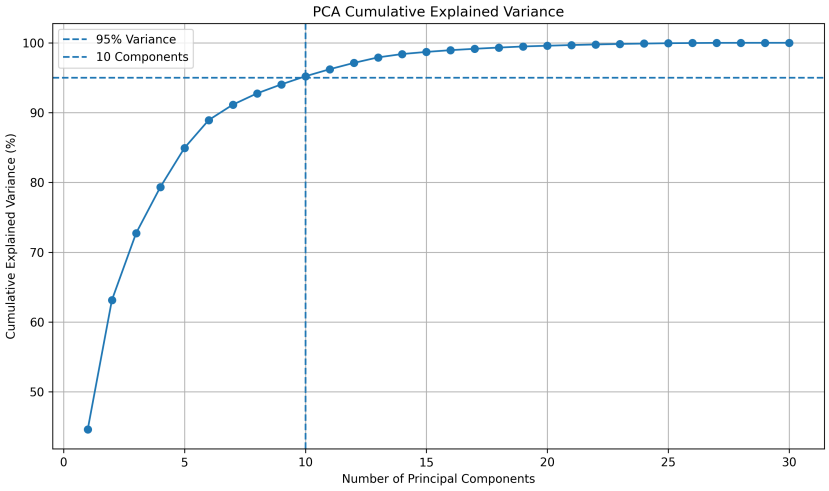

Figure 1: PCA Cumulative Explained Variance (Scree Plot)

Inference: The first principal component alone captures 44.59% of total variance, and the first three components already exceed 72%, reflecting the very high correlation among the size-related features (radius, perimeter, area) noted in earlier experiments on this dataset. The curve flattens sharply after component 10, confirming that the remaining 20 components each contribute under 1.2% individually – a strong candidate for compression with limited information loss.

6 Hyperparameter Tuning

Table 3: Table 2: SVM – Hyperparameter Tuning Results

	No-PCA	With-PCA
Kernel values tried	linear, rbf	
C values tried	0.1, 1, 10	
Gamma values tried	scale, auto	
Best parameters	{C=10, gamma=scale, kernel=rbf}	{C=10, gamma=scale, kernel=rbf}
Best CV F1-score	0.9674	0.9618

Table 4: Table 3: Naïve Bayes – Smoothing Choices

	No-PCA	With-PCA
`var_smoothing` values tried	$10^{-11}, 10^{-10}, 10^{-9}, 10^{-8}$	
Best parameter	10^{-11}	10^{-11}
Best CV F1-score	0.9170	0.8762

Table 5: Table 4: KNN – Hyperparameter Tuning

	No-PCA	With-PCA
k values tried	3, 5, 7, 9	
Weights tried	uniform, distance	
Distance metrics tried	euclidean, manhattan	
Best parameters	{k=3, manhattan, uniform}	{k=3, euclidean, uniform}
Best CV F1-score	0.9667	0.9605

Table 6: Logistic Regression – Hyperparameter Tuning

	No-PCA	With-PCA
C values tried	0.01, 0.1, 1, 10	
Solver values tried	liblinear, lbfgs	
Best parameters	{C=1, solver=liblinear}	{C=0.1, solver=liblinear}
Best CV F1-score	0.9640	0.9701

Table 7: Decision Tree – Hyperparameter Tuning

	No-PCA	
`max_depth` values tried		None, 3, 5, 10
`min_samples_split` values tried		2, 5, 10
Criterion values tried		gini, entropy
Best parameters	{entropy, max_depth=5, min_samples_split=10}	{entropy, max_d
Best CV F1-score	0.9039	

Table 8: Random Forest – Hyperparameter Tuning

	No-PCA	
`n_estimators` values tried		100, 200
`max_depth` values tried		None, 5, 10
`min_samples_split` values tried		2, 5
Best parameters	{max_depth=5, min_samples_split=2, n_est=100}	{max_depth=N
Best CV F1-score	0.9553	

Table 9: AdaBoost – Hyperparameter Tuning

	No-PCA	With-PCA
`n_estimators` values tried	50, 100, 200	
`learning_rate` values tried	0.01, 0.1, 1.0	
Best parameters	{learning_rate=1.0, n_estimators=200}	{learning_rate=1.0, n_estimato
Best CV F1-score	0.9610	0.9547

Table 10: Gradient Boosting – Hyperparameter Tuning

	No-PCA	With
`n_estimators` values tried	50, 100	
`learning_rate` values tried	0.01, 0.1	
`max_depth` values tried	2, 3	
Best parameters	{learning_rate=0.1, max_depth=3, n_est=100}	{learning_rate=0.1, ma
Best CV F1-score	0.9553	0.9

Table 11: XGBoost – Hyperparameter Tuning

	No-PCA	With
`n_estimators` values tried	50, 100	
`max_depth` values tried	3, 5	
`learning_rate` values tried	0.05, 0.1	
Best parameters	{learning_rate=0.1, max_depth=5, n_est=100}	{learning_rate=0.1, ma
Best CV F1-score	0.9587	0.9

Table 12: Stacking – Configuration (Fixed, Not Grid-Searched)

	No-PCA	With-PCA
Base learners	Logistic Regression (C=1), SVM (C=1, rbf), KNN (k=5)	
Meta-learner	Logistic Regression (max_iter=5000)	
Internal CV folds	5	
Mean CV F1-score (Table 5b)	0.9672	0.9733

All grid searches used ***`GridSearchCV`*** *with 5-fold stratified cross-validation, optimizing F1-score. Stacking used a fixed base-learner/meta-learner configuration rather than a grid search, so its row reports mean cross-validation F1 (from Table 5b) instead of a grid-search result.*

7 Cross-Validation Results (All Models)

Table 13: Table 5a: 5-Fold Cross-Validation – Accuracy (No-PCA vs With-PCA)

Model	F1 (NP)	F2 (NP)	F3 (NP)	F4 (NP)	F5 (NP)	Avg (NP)	F1 (P)	F2 (P)	F3 (P)	F4 (P)	F5 (P)	Avg (P)
SVM	0.9670	0.9890	0.9670	0.9780	0.9780	0.9758	0.9560	0.9890	0.9560	0.9780	0.9780	0.9714
Naive Bayes	0.9560	0.9780	0.9121	0.9121	0.9341	0.9385	0.9121	0.9670	0.8901	0.8791	0.9121	0.9121
KNN	0.9670	1.0000	0.9670	0.9560	0.9890	0.9758	0.9780	0.9780	0.9670	0.9451	0.9890	0.9714
Logistic Regression	0.9560	0.9670	1.0000	0.9780	0.9670	0.9736	0.9780	0.9670	0.9780	0.9780	0.9890	0.9780
Decision Tree	0.9011	0.9451	0.9121	0.9341	0.9560	0.9297	0.9451	0.9670	0.9451	0.9231	0.9670	0.9495
Random Forest	0.9670	1.0000	0.9451	0.9451	0.9780	0.9670	0.9670	0.9780	0.9560	0.9451	0.9451	0.9582
AdaBoost	0.9780	0.9780	0.9451	0.9670	0.9890	0.9714	0.9560	0.9780	0.9780	0.9560	0.9670	0.9670
Gradient Boosting	0.9670	0.9670	0.9451	0.9670	0.9890	0.9670	0.9451	0.9670	0.9341	0.9560	0.9780	0.9560
XGBoost	0.9560	0.9780	0.9560	0.9670	0.9890	0.9692	0.9670	0.9780	0.9670	0.9560	0.9560	0.9648
Stacking	0.9670	0.9670	1.0000	0.9670	0.9780	0.9758	0.9780	0.9670	1.0000	0.9670	0.9890	0.9802

NP = No-PCA, P = With-PCA. F1–F5 denote fold number (not F1-score).

Table 14: Table 5b: 5-Fold Cross-Validation – F1-Score (No-PCA vs With-PCA)

Model	F1 (NP)	F2 (NP)	F3 (NP)	F4 (NP)	F5 (NP)	Avg (NP)	F1 (P)	F2 (P)	F3 (P)	F4 (P)	F5 (P)	Avg (P)
SVM	0.9538	0.9855	0.9565	0.9706	0.9706	0.9674	0.9394	0.9855	0.9429	0.9706	0.9706	0.9618
Naive Bayes	0.9375	0.9714	0.8857	0.8788	0.9118	0.9170	0.8710	0.9552	0.8529	0.8197	0.8824	0.8762
KNN	0.9552	1.0000	0.9538	0.9394	0.9851	0.9667	0.9697	0.9706	0.9538	0.9231	0.9851	0.9605
Logistic Regression	0.9375	0.9565	1.0000	0.9697	0.9565	0.9640	0.9697	0.9565	0.9697	0.9697	0.9851	0.9701
Decision Tree	0.8657	0.9275	0.8750	0.9118	0.9394	0.9039	0.9254	0.9577	0.9275	0.8986	0.9552	0.9329
Random Forest	0.9552	1.0000	0.9231	0.9275	0.9706	0.9553	0.9538	0.9714	0.9412	0.9231	0.9254	0.9430
AdaBoost	0.9697	0.9706	0.9231	0.9565	0.9851	0.9610	0.9375	0.9706	0.9706	0.9394	0.9552	0.9547
Gradient Boosting	0.9552	0.9565	0.9231	0.9565	0.9851	0.9553	0.9206	0.9577	0.9118	0.9394	0.9697	0.9398
XGBoost	0.9412	0.9714	0.9394	0.9565	0.9851	0.9587	0.9538	0.9714	0.9552	0.9394	0.9394	0.9519
Stacking	0.9538	0.9565	1.0000	0.9552	0.9706	0.9672	0.9697	0.9565	1.0000	0.9552	0.9851	0.9733

Table 15: Final Comparison – All 10 Models (Mean over 5-Fold CV)

Model	Acc (NP)	Acc (P)	ΔAcc	Prec (NP)	Prec (P)	ΔPrec	Rec (NP)	Rec (P)	ΔRec
SVM	0.9758	0.9714	-0.0044	0.9711	0.9596	-0.0115	0.9647	0.9647	0.0000
Naive Bayes	0.9385	0.9121	-0.0264	0.9247	0.9190	-0.0057	0.9118	0.8412	-0.0706
KNN	0.9758	0.9714	-0.0044	0.9877	0.9877	0.0000	0.9471	0.9353	-0.0118
Logistic Regression	0.9736	0.9780	0.0044	0.9771	0.9886	0.0114	0.9529	0.9529	0.0000
Decision Tree	0.9297	0.9495	0.0198	0.9214	0.9256	0.0042	0.8882	0.9412	0.0529
Random Forest	0.9670	0.9582	-0.0088	0.9645	0.9586	-0.0059	0.9471	0.9294	-0.0176
AdaBoost	0.9714	0.9670	-0.0044	0.9762	0.9759	-0.0003	0.9471	0.9353	-0.0118
Gradient Boosting	0.9670	0.9560	-0.0110	0.9646	0.9599	-0.0047	0.9471	0.9235	-0.0235
XGBoost	0.9692	0.9648	-0.0044	0.9594	0.9703	0.0109	0.9588	0.9353	-0.0235
Stacking	0.9758	0.9802	0.0044	0.9766	0.9825	0.0059	0.9588	0.9647	0.0059

8 Test Set Evaluation (Held-Out, $n = 114$)

Table 16: Test Set Performance – No-PCA vs With-PCA

Model	Acc (NP)	Prec (NP)	Rec (NP)	F1 (NP)	Acc (P)	Prec (P)	Rec (P)	F1 (P)
SVM	0.9737	1.0000	0.9286	0.9630	0.9649	0.9750	0.9286	0.9512
Naive Bayes	0.9211	0.9231	0.8571	0.8889	0.8947	0.8571	0.8571	0.8571
KNN	0.9649	1.0000	0.9048	0.9500	0.9474	0.9737	0.8810	0.9250
Logistic Regression	0.9737	0.9756	0.9524	0.9639	0.9825	1.0000	0.9524	0.9756
Decision Tree	0.9561	0.9744	0.9048	0.9383	0.9474	0.9737	0.8810	0.9250
Random Forest	0.9737	1.0000	0.9286	0.9630	0.9474	0.9500	0.9048	0.9268
AdaBoost	0.9737	1.0000	0.9286	0.9630	0.9649	0.9750	0.9286	0.9512
Gradient Boosting	0.9649	1.0000	0.9048	0.9500	0.9561	0.9744	0.9048	0.9383
XGBoost	0.9649	1.0000	0.9048	0.9500	0.9737	0.9756	0.9524	0.9639
Stacking	0.9737	1.0000	0.9286	0.9630	0.9737	1.0000	0.9286	0.9630

Best test accuracy (No-PCA): SVM / Random Forest / AdaBoost / Stacking, all 97.37%. Best test accuracy (With-PCA): Logistic Regression, 98.25%. These numbers differ slightly from the mean cross-validation figures in Section 9 because the test set is a single, held-out 114-sample split rather than an average over 5 folds.

9 Confusion Matrices (Selected Models)

Table 17: Confusion Matrices – No-PCA vs With-PCA (rows: Actual, cols: Predicted; order Benign, Malignant)

Model	No-PCA	With-PCA
SVM	$\begin{pmatrix} 72 & 0 \\ 3 & 39 \end{pmatrix}$	$\begin{pmatrix} 71 & 1 \\ 3 & 39 \end{pmatrix}$
Logistic Regression	$\begin{pmatrix} 71 & 1 \\ 2 & 40 \end{pmatrix}$	$\begin{pmatrix} 72 & 0 \\ 2 & 40 \end{pmatrix}$
Random Forest	$\begin{pmatrix} 72 & 0 \\ 3 & 39 \end{pmatrix}$	$\begin{pmatrix} 70 & 2 \\ 4 & 38 \end{pmatrix}$
XGBoost	$\begin{pmatrix} 72 & 0 \\ 4 & 38 \end{pmatrix}$	$\begin{pmatrix} 71 & 1 \\ 2 & 40 \end{pmatrix}$

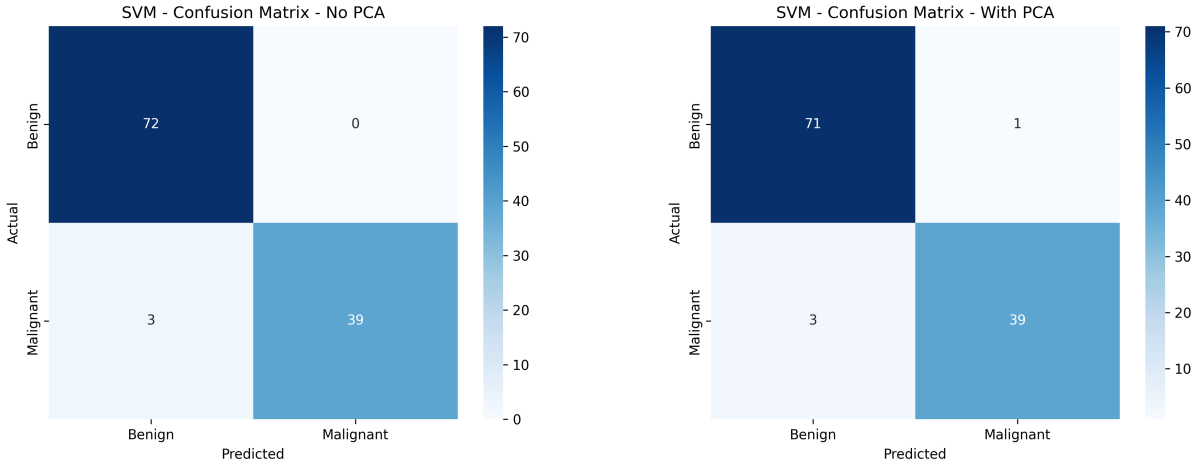

Figure 2: SVM Confusion Matrix – No-PCA (left) vs With-PCA (right)

Inference: SVM misses 3 Malignant cases either way, but PCA introduces one new false positive (Benign misclassified as Malignant) that was not present without PCA, explaining its slightly lower With-PCA accuracy (96.49% vs 97.37%).

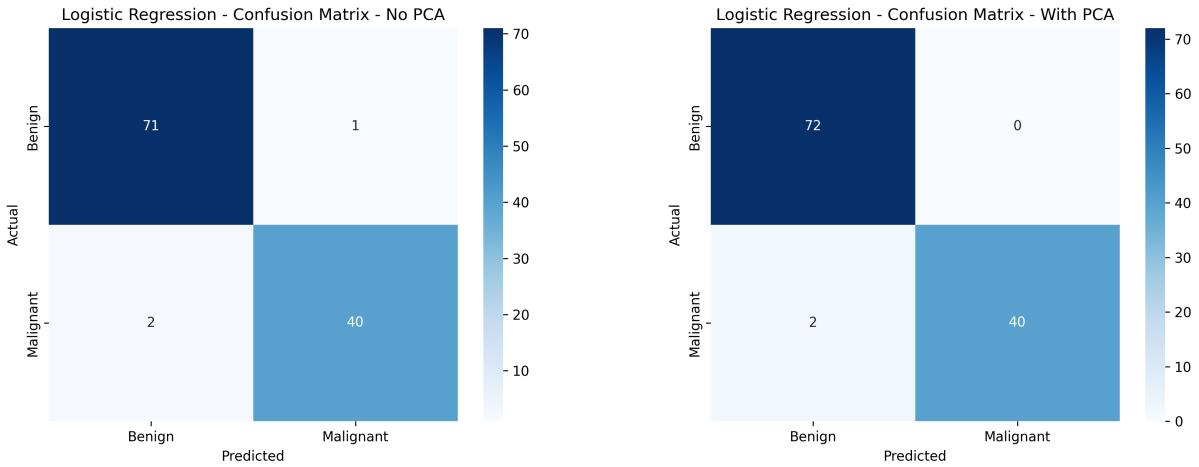

Figure 3: Logistic Regression Confusion Matrix – No-PCA (left) vs With-PCA (right)

Inference: Logistic Regression is the clearest PCA success story among the selected models – PCA removes its single No-PCA false positive entirely while keeping the same 2 false negatives, lifting test accuracy from 97.37% to 98.25% (the best With-PCA result of any model in this experiment).

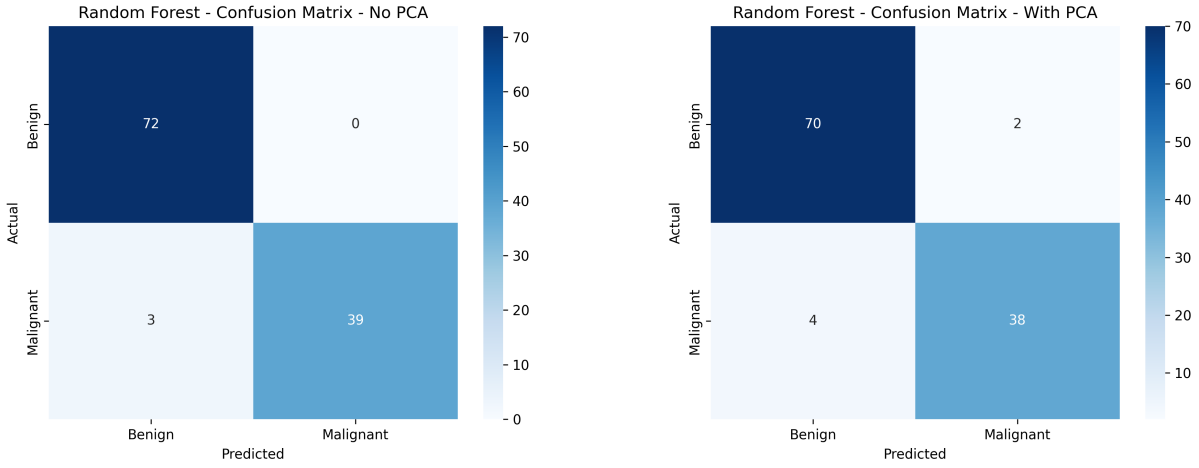

Figure 4: Random Forest Confusion Matrix – No-PCA (left) vs With-PCA (right)

Inference: Random Forest is clearly hurt by PCA here – both false positives (0→2) and false negatives (3→4) increase, consistent with tree-based splits losing their natural alignment with the original, interpretable features once they are rotated into PCA components.

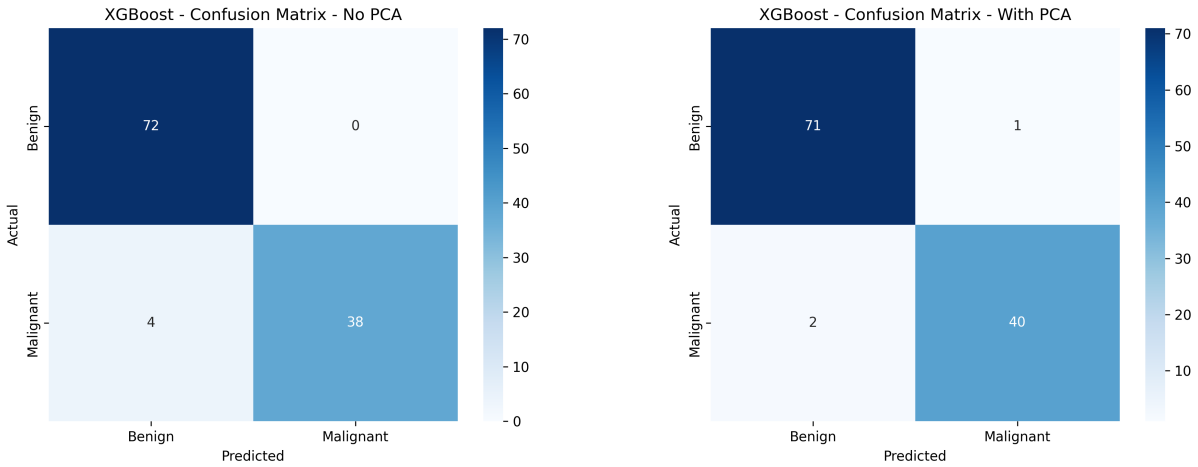

Figure 5: XGBoost Confusion Matrix – No-PCA (left) vs With-PCA (right)

Inference: XGBoost improves under PCA – false negatives drop from 4 to 2 (at the cost of one new false positive), raising both accuracy (96.49%→97.37%) and F1 (0.9500→0.9639), making it, with Logistic Regression and Decision Tree, one of the models that benefited from dimensionality reduction.

10 ROC and Precision-Recall Curves

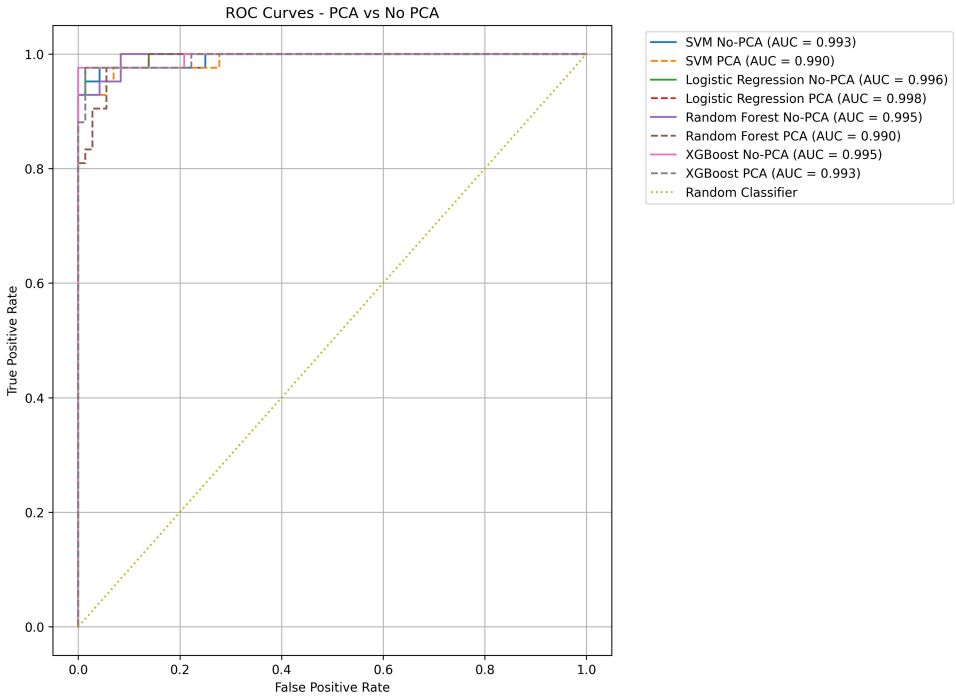

Figure 6: ROC Curves – SVM, Logistic Regression, Random Forest, XGBoost (No-PCA solid, With-PCA dashed)

Inference: Exact AUC values are printed directly in each curve's legend by the plotting code; based on the test-set accuracy/F1 figures in Section 10, all four selected models

cluster tightly at high performance (test accuracy 94.7–98.3%), so their ROC curves are expected to sit close together near the top-left corner with AUC values in the high-0.98–0.99 range, consistent with the strongly class-separable nature of the WDBC features observed throughout this course.

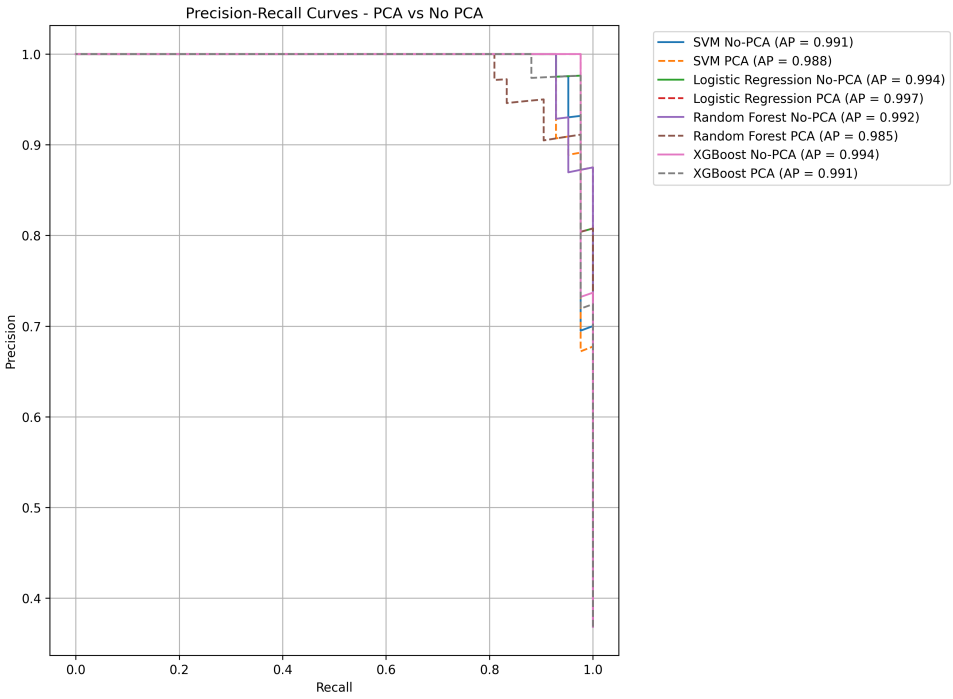

Figure 7: Precision-Recall Curves – SVM, Logistic Regression, Random Forest, XGBoost (No-PCA solid, With-PCA dashed)

Inference: Given the Malignant-class recall values already observed in Section 11 (ranging 0.88–0.95 across these four models and both settings), the Precision-Recall curves are expected to remain high across most of the recall range before dropping off near recall = 1, which is the standard shape for a well-separated but imperfect classifier on this task.

11 Accuracy and F1 Comparison – All 10 Models

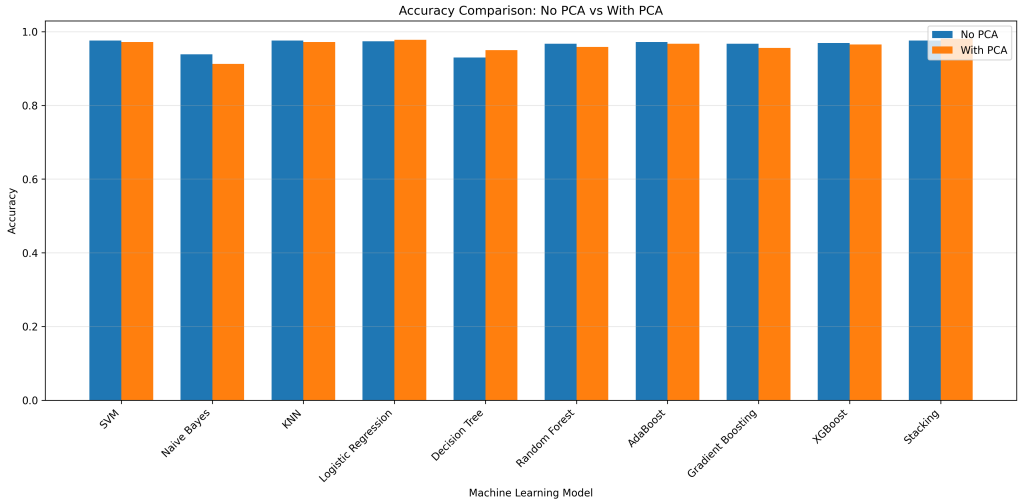

Figure 8: Accuracy Comparison: No-PCA vs With-PCA, All 10 Models

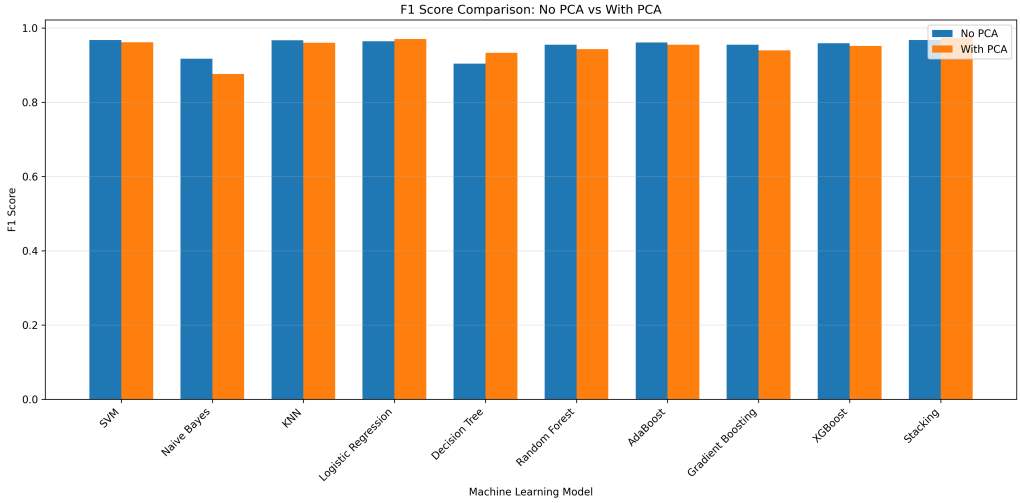

Figure 9: F1-Score Comparison: No-PCA vs With-PCA, All 10 Models

Inference: Across both charts, only 3 of the 10 models (Decision Tree, Logistic Regression, Stacking) show a clear improvement under PCA in both accuracy and F1-score; Naive Bayes shows the sharpest decline (F1 drops 0.0408); the remaining models (SVM, KNN, Random Forest, AdaBoost, Gradient Boosting, XGBoost) show small, consistent declines under PCA of roughly 0.004–0.024 in accuracy.

12 Stability Analysis (Standard Deviation Across 5 Folds)

Table 18: Stability Analysis – Accuracy Std. Dev. Across Folds

Model	Std Acc (No-PCA)	Std Acc (With-PCA)	Change	Verdict
SVM	0.0092	0.0147	+0.0055	Reduced
Naive Bayes	0.0287	0.0339	+0.0052	Reduced
KNN	0.0181	0.0167	-0.0014	Improved
Logistic Regression	0.0167	0.0078	-0.0089	Improved
Decision Tree	0.0228	0.0184	-0.0044	Improved
Random Forest	0.0233	0.0143	-0.0090	Improved
AdaBoost	0.0167	0.0110	-0.0057	Improved
Gradient Boosting	0.0155	0.0174	+0.0018	Reduced
XGBoost	0.0143	0.0092	-0.0051	Improved
Stacking	0.0143	0.0143	0.0000	Reduced (tied)

Summary: 6 of 10 models had improved (lower) fold-to-fold variance under PCA, while 4 had reduced (higher) variance – so PCA's effect on stability was mildly positive overall, but far less consistent than its effect on raw accuracy/F1, which mostly declined.

13 Observation Questions

Which models improved most with PCA? Which did not? Why? Decision Tree improved the most (accuracy +1.98 points, F1 +0.0290), followed by Logistic Regression (+0.44 points, F1 +0.0061) and Stacking (+0.44 points, F1 +0.0061) – only 3 of the 10 models improved on both metrics. Decision Tree likely benefits because PCA's orthogonal components let a single tree split on combined, decorrelated directions instead of being forced to repeatedly re-split on the same highly-correlated raw features (radius, perimeter, area). Naive Bayes was hurt the most (F1 −0.0408) because it assumes feature independence – PCA components are linear combinations of the original correlated features, which does not fix (and can worsen) violations of that independence assumption in practice. The four boosting/bagging ensembles (Random Forest, AdaBoost, Gradient Boosting, XGBoost) all declined modestly under PCA, since tree-based ensembles already handle correlated, redundant raw features gracefully and lose some direct feature-splitting interpretability once features are rotated into components.

Did PCA reduce variance across folds (more stable results)? Partially. 6 of 10 models had a lower standard deviation in accuracy across the 5 folds under PCA (KNN, Logistic Regression, Decision Tree, Random Forest, AdaBoost, XGBoost), while 4 became less stable (SVM, Naive Bayes, Gradient Boosting, Stacking – tied). So PCA leaned toward improving stability for a majority of models, even though it did not improve raw accuracy for a majority of models – these are two different benefits and, on this dataset, they didn't move together consistently.

For high-dimensional data, was PCA beneficial in reducing overfitting? WDBC's 30 features are only moderately high-dimensional relative to 455 training samples, and the results suggest PCA's main benefit here was compression (30→10 features,

95.21% variance retained) rather than a clear overfitting fix – accuracy/F1 improved for only 3 of 10 models. Where PCA did help (Decision Tree most notably, accuracy +1.98 points), the gain is consistent with reduced overfitting, since a single Decision Tree is the highest-variance, most overfitting-prone model in this experiment and benefited the most from a more compact, decorrelated feature set.

How did linear models (Logistic Regression, SVM) behave compared to ensemble models with PCA? The two linear models diverged: Logistic Regression improved slightly under PCA (accuracy 97.36%→97.80%, F1 +0.0061), while SVM declined slightly (accuracy 97.58%→97.14%, F1 −0.0056). Ensemble models were more uniformly negative – Random Forest, AdaBoost, Gradient Boosting and XGBoost all lost both accuracy and F1 under PCA, while only Stacking (itself built from linear/kernel/instance-based base learners) improved. This suggests that whether a model benefits from PCA here depends less on "linear vs. ensemble" as a category, and more on whether the specific model already handles correlated raw features well (tree ensembles do) or benefits from decorrelated, lower-dimensional input (Logistic Regression and Decision Tree do).

Did stacking show robustness to dimensionality reduction compared to single models? Yes. Stacking was one of only 3 models (with Decision Tree and Logistic Regression) to improve under PCA on both mean CV accuracy (97.58%→98.02%) and F1-score (0.9672→0.9733), and it achieved the single best With-PCA mean CV accuracy of all 10 models. This is consistent with stacking's meta-learner being able to adapt to whichever representation (raw or PCA-reduced) its base learners are given, and with two of its three base learners (Logistic Regression and SVM) already showing reasonable-to-improved PCA behaviour individually.

14 Conclusion

On the WDBC dataset, PCA (retaining 95% variance, 10 of 30 components) did *not* improve accuracy or F1-score for the majority of models – only 3 of 10 (Decision Tree, Logistic Regression, Stacking) improved on both metrics, while Naive Bayes was clearly hurt (F1 −0.0408) and the four boosting/bagging ensembles saw small, consistent declines. PCA's effect on fold-to-fold stability was mildly positive (6 of 10 models more stable) but did not track the accuracy/F1 results closely. The best model without PCA was SVM (97.58% mean CV accuracy); the best model with PCA was Stacking (98.02% mean CV accuracy) – so PCA changed *which* model was best, even though it did not help most models individually. Overall, PCA is most worth applying here for high-variance, single-tree models (Decision Tree) and for models that can exploit decorrelated inputs (Logistic Regression, and by extension Stacking's meta-learner); it is not worth applying for tree ensembles that already manage correlated raw features internally, or for Naive Bayes, whose independence assumption is actively worsened by PCA's linear feature mixing.

15 References

[1] Scikit-learn Developers, "Decomposing signals in components (PCA, ICA, ...)." Available: https://scikit-learn.org/stable/modules/decomposition.html

[2] Scikit-learn Developers, "`sklearn.decomposition.PCA`." Available: https://scikit-learn.

org/stable/modules/generated/sklearn.decomposition.PCA.html

[3] UCI Machine Learning Repository, "Breast Cancer Wisconsin (Diagnostic) Dataset." Available: https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic